LAB 0 • STUDENT GUIDE • SESSION 1 / BEFORE LAB 1

Meet the Data

"What is the model shown, and what decides pass or fail?"

Trusting AI-Written Code • AeroEval edition • HumanEval and AeroEval • v1.0, November 2026

1. Lab 0 at a glance

Item	Detail
Why	Labs 1 and 2 grade AI-written code against two datasets. Before you judge any score, you should know what a problem looks like, what the AI model is shown, and what decides pass or fail.
What you do	Open both datasets in a Google Colab notebook and explore them with tables and figures.
Time	About 25 minutes.
You need	A Google account and three files: `lab0_s1.ipynb`, `humaneval_sample.json`, `aero20_s1.json`. No API key.
The AI model	Twice, the notebook asks **Colab's free built-in AI model** one problem — one from HumanEval, one from AeroEval — and grades the answer live. No key or payment; you must be signed in to Colab.
What else runs	The datasets' own reference code, and a few deliberately wrong teaching examples that are always labelled as such.

2. Run it in Google Colab

1. Go to `colab.research.google.com` and sign in. In the dialog, choose the **Upload** tab (or **File** → **Upload notebook**) and select `lab0_s1.ipynb`.
2. Click the **folder icon** in the left sidebar to open the Files panel. Click the **upload icon** and select `humaneval_sample.json` and `aero20_s1.json`. Wait until both appear in the list. (Colab warns that uploaded files are deleted when the session ends; that is expected.)
3. Choose **Runtime** → **Run all**. If Colab says the notebook was not written by Google, click **Run anyway**.
4. It finishes in a minute or two (the two model calls take a few seconds each). Then read it from the top: every step has a short explanation above it.

If something goes wrong

`FileNotFoundError` **in Step 3:** a JSON file is missing or its name has changed (for example `aero20_s1 (1).json`). Re-upload it with the exact name. **Files gone after a break:** Colab resets idle sessions; upload the two files again and Run all. **"The model is not available here":** you are not in Colab, not signed in, or the free model is busy; wait a minute and re-run that cell — every other step still works. **Anything else: Runtime → Restart session and run all**.

3. What the notebook walks you through

Step	What you see
1–3	Setup, settings, and loading the two files
4	**HumanEval:** one problem field by field; the ten sample problems and a chart of their test strength
5	How HumanEval decides pass or fail; an optional check that the sample matches the official dataset
Aha	A wrong, hard-coded answer that passes a HumanEval test
6	**Ask the free model** HumanEval/0: the prompt sent, the raw reply, the code, the verdict
7	**AeroEval:** the file's structure, and all ten problems with the reference solutions re-run
8	One problem (AE16, stall speed) in full; minimal vs full prompt lengths
9	**Ask the free model** AE16: its numbers vs visible tests, held-out cases and sanity checks
10	The planted defects and whether tests and sanity checks catch them
Aha	A planted defect whose curve looks plausible but is wrong
11	HumanEval and AeroEval side by side; check-your-understanding questions
Try it	Ask the free model any AeroEval problem, with the minimal or full prompt

4. Part A — HumanEval

HumanEval is OpenAI's public benchmark of 164 short Python problems (Chen et al., 2021). Many headline "pass rate" numbers for code models are HumanEval scores. Each problem gives a function signature and a docstring; the model writes the body; a hidden test decides pass or fail. Your sample has ten problems (license: MIT (openai/human-eval)).

The five fields of every record:

Field	What it is
`task_id`	unique id, HumanEval/¡n¿
`prompt`	imports, signature and docstring the model is given
`entry_point`	name of the function to test
`canonical_solution`	a reference function body (indented)
`test`	Python code defining check(candidate); passing = no assertion fails

What the model sees is only the `prompt`. Here is HumanEval/0's:

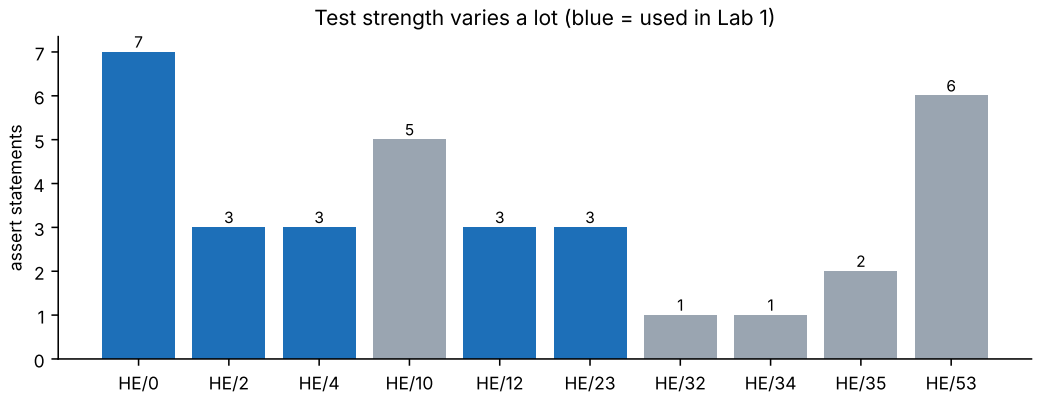
```
from typing import List


def has_close_elements(numbers: List[float], threshold: float) -> bool:
    """ Check if in given list of numbers, are any two numbers closer to each other than
    given threshold.
    >>> has_close_elements([1.0, 2.0, 3.0], 0.5)
    False
    >>> has_close_elements([1.0, 2.8, 3.0, 4.0, 5.0, 2.0], 0.3)
    True
    """
```

How pass or fail is decided: the notebook joins `prompt` + the model's body + `test`, then calls `check(entry_point)`. If no `assert` fails, the problem passes. Most answers are compared exactly (`==`); a tolerance appears only where the test's author wrote one.

Your ten problems. Five are the ones Lab 1 uses; the other five show how varied the tests are.

Problem	Function	Asserts	Lab 1	Why it is in the sample
HumanEval/0	has_close_elements	7	yes	Used in Lab 1. A typical problem: signature, docstring with examples, 7 asserts.
HumanEval/2	truncate_number	3	yes	Used in Lab 1. Floating-point answers checked with a tolerance (abs(...) ¡ 1e-6).
HumanEval/4	mean_absolute_deviation	3	yes	Used in Lab 1. Numeric result, tolerance-based tests.
HumanEval/10	make_palindrome	5		The prompt contains a helper function (is_palindrome) the solution must use.
HumanEval/12	longest	3	yes	Used in Lab 1. Returns None for empty input; the docstring example prints nothing.
HumanEval/23	strlen	3	yes	Used in Lab 1. The simplest kind of problem: one line of code.
HumanEval/32	find_zero	1		Helper in the prompt; the test is a seeded random loop checking a property (—poly(x)— ¡ 1e-4), not fixed answers.
HumanEval/34	unique	1		A single assert: the weakest possible test. A hard-coded answer passes it.
HumanEval/35	max_element	2		Only 2 asserts, and the test uses 124 where the docstring example uses 123.
HumanEval/53	add	6		An UNSEEDED random test: two runs check different inputs, so the test is not reproducible.



Assertions per test (blue = used in Lab 1). A loop counts once; HE/32 and HE/53 loop over random inputs.

Aha 1 — a wrong answer that passes

HumanEval/34 asks for the sorted unique elements of *any* list. Its test checks a single input. This body (*written by the instructor to make the point — not a model's output*) ignores its input and returns that one answer:

```
return [0, 2, 3, 5, 9, 123]
```

HumanEval's own test says **pass**. But `unique([3, 1, 3])` returns `[0, 2, 3, 5, 9, 123]` instead of `[1, 3]`. **A score is only as good as its tests.** EvalPlus (HumanEval+) added many tests per problem and found solutions that passed the originals but were wrong.

5. Ask the free model: a HumanEval problem (Step 6)

The notebook sends HumanEval/0 to Colab's free model — exactly this prompt, the same one Lab 1 uses:

````
Complete this Python function. Reply with the whole function in one ```python block.

from typing import List


def has_close_elements(numbers: List[float], threshold: float) -> bool:
    """ Check if in given list of numbers, are any two numbers closer to each other than
    given threshold.
    >>> has_close_elements([1.0, 2.0, 3.0], 0.5)
    False
    >>> has_close_elements([1.0, 2.8, 3.0, 4.0, 5.0, 2.0], 0.3)
    True
    """
````

You then see four things, in order: **(1)** the prompt, **(2)** the model's raw reply, unedited, **(3)** the code taken out of the reply, and **(4)** HumanEval's own `check()` verdict. The model's code runs in a separate process; the verdict is decided in the notebook.

What a PASS does and does not tell you

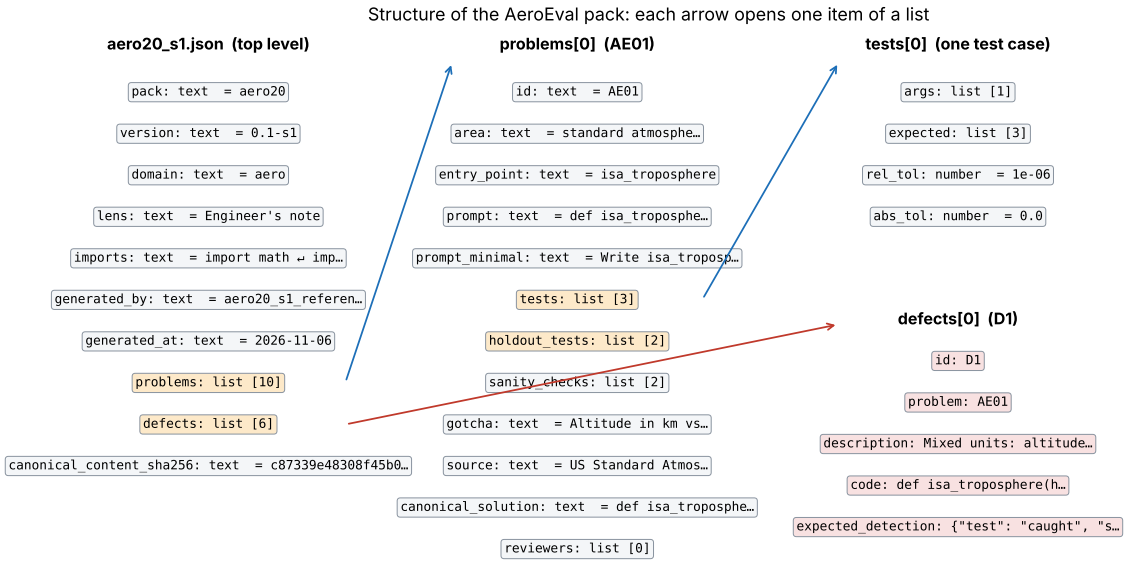
It is a **live** answer: run the cell again and the reply may differ, and your neighbour's may differ from yours. A PASS means this one answer survived this problem's seven assertions — nothing more. Whether the model is reliable needs many problems, repeated tries and honest uncertainty: that is Lab 1.

6. Part B — AeroEval

AeroEval (`aero20_s1.json`) is the course's aerospace problem pack: 10 problems, each a formula or solver with units and conventions, plus 6 planted wrong versions for testing the tests. Labs 1 and 2 use exactly this file.

Top level of the file:

Key	Type	What it holds
`pack`	text	the pack's name
`version`	text	pack version
`domain`	text	edition (aero)
`lens`	text	label for engineering notes
`imports`	text	imports every solution runs with
`generated_by`	text	the script that built and verified the file
`generated_at`	text	when it was built
`problems`	list	the 10 problems
`defects`	list	6 planted wrong versions
`canonical_content_sha256`	text	a fingerprint of the content



The structure: each arrow opens one item of a list — the top level, one problem, one test case, and one defect.

Every problem has these fields:

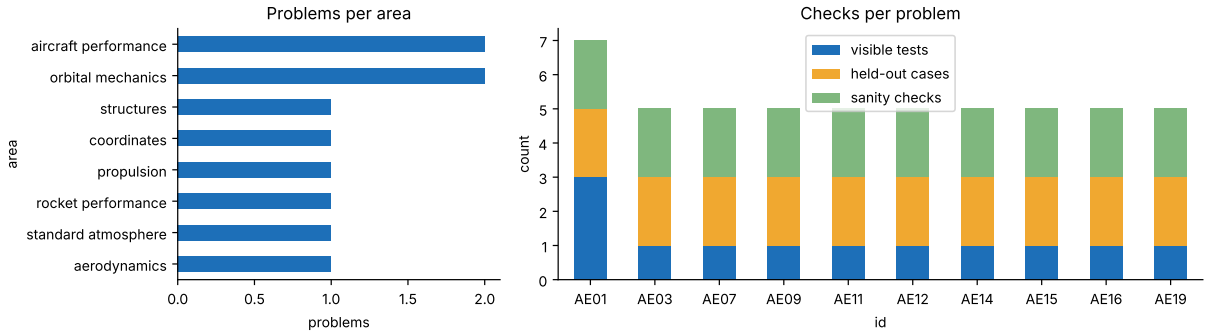
Field	What it is
`id`	AE number
`area`	engineering area
`entry_point`	function name
`prompt`	the full prompt: signature + docstring with units, constants and assumptions
`prompt_minimal`	a one-line prompt that leaves the details out
`tests`	visible test cases (may appear in a prompt)
`holdout_tests`	held-out cases: never shown in any prompt
`sanity_checks`	physical-property checks (code)
`gotcha`	the classic slip on this problem
`source`	where the physics comes from
`canonical_solution`	the reference implementation
`reviewers`	review sign-offs

Each test case has `args` (the inputs), `expected` (the reference answer), and two tolerances, `rel_tol` and `abs_tol`. An answer passes when it is within $\max(\text{rel_tol} \cdot |\text{expected}|,\ \text{abs_tol})$ of the expected value. For example, AE16 (stall speed) with inputs [1100, 1.225, 16.2, 1.6] expects 26.0667 m/s, within a relative tolerance of 1e-06.

Visible tests and held-out cases. The pack has 12 visible tests and 20 held-out cases (two per problem). A visible test may be shown to the model — Lab 2's "tests-first" prompt includes them. A held-out case is *never* put in any prompt, so it shows whether code works beyond the examples it saw. It is withheld from the prompt, not secret: it sits in this file.

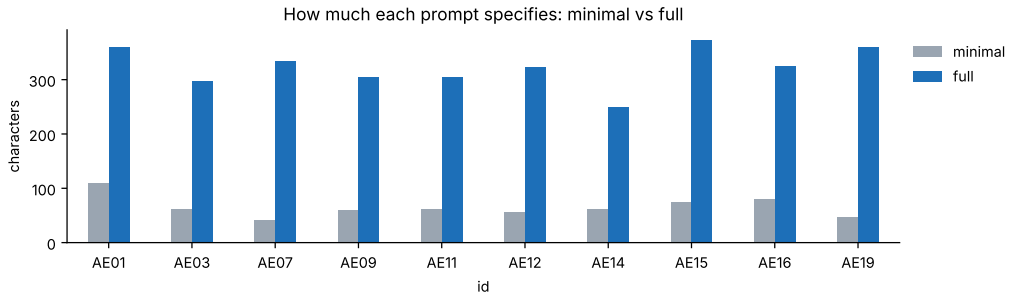
The ten problems:

ID	Area	Function	Tests	Held-out	The classic slip
AE01	standard atmosphere	`isa_troposphere`	3	2	Altitude in km vs m; geometric vs geopotential altitude.
AE03	aerodynamics	`max_lift_to_drag`	1	2	Oswald factor omitted from k.
AE07	propulsion	`rocket_thrust`	1	2	Pressure term dropped or sign flipped.
AE09	rocket performance	`tsiolkovsky_dv`	1	2	log10 instead of natural log; Isp in s vs exhaust velocity.
AE11	orbital mechanics	`circular_orbit`	1	2	Altitude used as radius; mu in m{}3/s{}2 vs km{}3/s{}2.
AE12	orbital mechanics	`solve_kepler`	1	2	Fixed iteration count; no convergence check; degrees vs radians.
AE14	coordinates	`quat_rotate`	1	2	Scalar-last order assumed (SciPy's default).
AE15	aircraft performance	`breguet_range_jet`	1	2	TSFC per hour used as per second.
AE16	aircraft performance	`stall_speed`	1	2	Mass passed where weight is required.
AE19	structures	`cantilever_tip_deflection`	1	2	Width and depth swapped in I; GPa vs Pa.



Problems per area, and the checks each problem has.

Minimal vs full prompt. Every problem has two prompts. The *full* prompt states units, constants and assumptions; the *minimal* prompt is one line and leaves the model to guess. Lab 1 compares the two.



Characters in each problem's minimal and full prompt.

Ask the free model: an AeroEval problem (Step 9). The notebook sends AE16 (stall speed) with its *full* prompt — the same prompt Lab 1 uses:

```
Write the Python function below. Reply with the code only, in one ```python block.

def stall_speed(mass_kg, rho, S, CLmax):
    """Return stall speed in m/s. mass_kg is aircraft MASS in kg (weight = mass * 9.80665
        N);
    rho is air density in kg/m^3; S is wing area in m^2; CLmax is the maximum lift
        coefficient
    for the configuration. Assumes steady level flight (lift = weight) and positive
        inputs."""
```

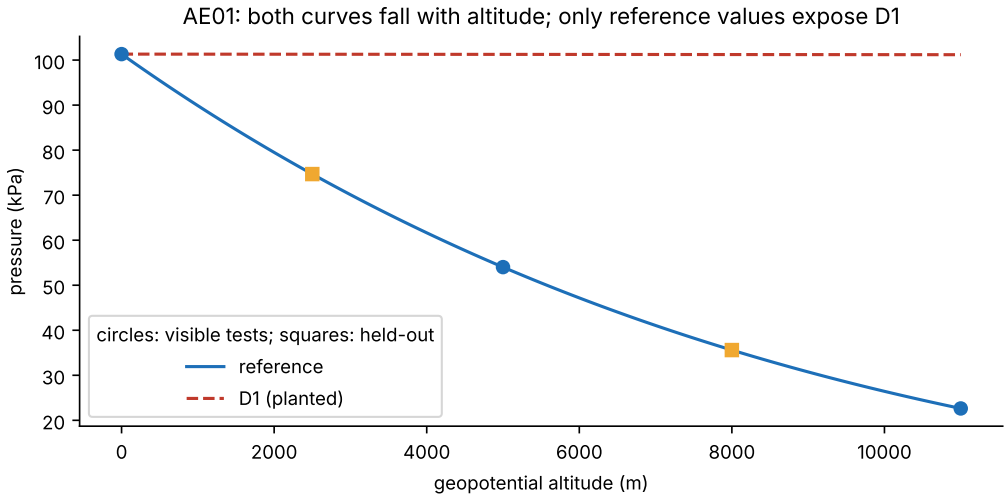
The model's function is run on the pack's visible tests, its held-out cases (never shown to the model) and its sanity checks, and a table shows the model's numbers next to the expected ones, with a one-line verdict. In the last cell (*Try it*) you can ask about any problem, with the minimal or the full prompt.

Planted defects. Each defect is a deliberately wrong version of one problem's code, written for teaching: *never a model's output, and never scored.* The pack records whether the visible reference test and the sanity checks catch it.

Defect	Problem	What is wrong	Test	Sanity
D1	AE01	Mixed units: altitude converted to km but the lapse rate left per metre (5000 m is computed as 5 m)	caught	MISSED
D3	AE16	Mass used as weight	caught	caught
D4	AE15	TSFC per hour used as per second	caught	caught
D5	AE14	Scalar-first quaternion read as scalar-last	caught	MISSED
D7	AE09	log10 instead of natural log	caught	MISSED
D12	AE12	Fixed three iterations, no convergence check	MISSED	caught

Aha 2 — a plausible curve at the wrong scale

Defect D1 converts altitude to kilometres but keeps the lapse rate per metre, so 5,000 m is computed as if it were 5 m. The pressure still falls with height, so a "pressure decreases" check passes — but at 5,000 m it gives about 101,265 Pa instead of 54,020 Pa. Only a reference value at a known altitude exposes it.



AE01: the reference curve and the planted D1. Circles are visible tests; squares are held-out cases.

7. HumanEval and AeroEval side by side

	HumanEval	**AeroEval**
What a problem is	a general programming puzzle	an aerospace formula or solver, with units
Pass or fail decided by	`check(candidate)`: assert statements	reference values within a stated tolerance
Comparison	mostly exact (`==`)	relative and absolute tolerance per test
Units and conventions	rarely matter	the main source of silent errors
Extra checks	none	sanity checks (physical properties)
Held-out cases	none	two per problem, never in a prompt
Planted defects	none	yes, to test the tests
Public?	yes, widely — models may have seen it	course-built — less likely to be memorised

8. Words to know

Term	Meaning in this course
Prompt	What the model is given: here, a function signature and docstring
Canonical solution	The dataset's reference implementation
Test / `check`	Code that decides pass or fail
Tolerance	How close a number must be to the expected value to count as right
Visible test	A test case that may be shown to the model
Held-out case	A case never shown in any prompt; checks that code works beyond its examples
Sanity check	A physical property the answer must have (for example, pressure falls with height)
Planted defect	A deliberately wrong version, used to test whether checks catch errors

9. Check your understanding

1. In HumanEval, what does the model see, and what decides pass or fail?
2. Why can a wrong answer pass HumanEval/34?
3. In AeroEval, what is the difference between a visible test and a held-out case? Is a held-out case secret?
4. Why does the D1 curve fool a "pressure falls with height" check?
5. AeroEval compares numbers with a tolerance, while most HumanEval tests use `==`. Why?
6. In Steps 6 and 9, what exactly did a PASS establish, and what did it not?

10. Try it on your own

In the notebook's last cell, set `ITEM` to a problem id and `VARIANT` to `"minimal"` or `"full"`, then run it to ask the free model. Try AE01 or AE14 with the minimal prompt: do the units and conventions survive? Then try the full prompt. Bring one observation to Lab 1 — and note which answers were live model output.

11. Answers

1. Only the prompt (signature and docstring). The hidden `check(candidate)` assertions decide.
2. Its test checks a single input, so returning that one answer passes.
3. A visible test may be shown to the model; a held-out case never is. It is withheld from prompts, not secret: it is in the file.
4. A consistent units error keeps the curve falling; only a reference value at a known altitude shows the scale is wrong.
5. Engineering answers are real numbers computed in floating point; two correct programs can differ in the last digits, so "close enough" must be defined — and the tolerance is part of the specification.
6. That one answer, on one run, matched these particular tests. Not that the model is reliable: a second run can differ, and one problem says nothing about the rest.

Next: Lab 1. You have seen what the model is shown, what decides pass or fail, and one live answer from each dataset. Lab 1 asks a model for every problem and measures what happens — with honest uncertainty.